DD: WO-01-FLOW-INSTALLATION — Installation Work Order Flow

Developer implementation guide: `DD_WO-01-FLOW-INSTALLATION-DEV_IMPL_GUIDE-v1.0.md`.

1. Purpose

This rewritten DD is the developer-facing version of the operator BPMN flow `DD_WO-01-FLOW-INSTALLATION-v1.0.md`.

Verdict: ■ New | **Wave:** 2 | **Group III:** Fulfillment & Operations | **Version:** v1.0 | **Module:** Work Order | **Satellite of:** DD_WO-01-Work_Order_Module-v1.0 The Installation flow satellite. Defines `WorkOrder_Installation_WIK.bpmn`, the Drools KJAR shape for assignment + finalization decisions, the operator-config rows seeding the framework's catalogs, the Pattern B "reassign-without-closure" mechanism for blocked installs, the equipment-binding capture at finalize, and the warranty + RPT linkage. Built on top of DD_WO-01-FRAMEWORK-v1.0; relies on its primitives without re-stating them. Covers all Job Types in the `INSTALLATION` kind: `H01` (HFC Installation), `G07` (VOIP Installation), `DLC` (Wi-Fi Extender Install — paid), `DL4` (Free Extender Install), `RTV` (Rescheduled TV Install), `EQU` (Equipment Upgrade). Plus the `RPT` linkage when a repeat-within-warranty WO is raised.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	`/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/07_work_order/DD_WO-01-FLOW-INSTALLATION-v1.0.md`
Version	`v1.0`
DD type	`operator BPMN flow`
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- `POST /api/work-orders`
- `POST /api/work-orders/wo_01HZ...B7/equipment-bindings`
- `POST /api/work-orders/wo_01HZ...B7/finalize-first-confirm`
- `POST /api/work-orders/wo_01HZ...B7/notes`
- `POST /api/work-orders/wo_01HZ...B7/reassign`

- POST /api/work-orders/wo_01HZ...T9/notes
- POST /api/work-orders/wo_01HZ...Z1/attachments
- POST /api/work-orders/wo_01HZ...Z1/claim
- POST /api/work-orders/wo_01HZ...Z1/equipment-bindings
- POST /api/work-orders/wo_01HZ...Z1/finalize-first-confirm
- POST /api/work-orders/wo_01HZ...Z1/finalize-second-confirm
- POST /api/work-orders/wo_01HZ...Z1/notes
- POST /api/work-orders/{woId}/attachments
- POST /api/work-orders/{woId}/equipment-bindings
- POST /api/work-orders/{woId}/topology-replay
- POST /attachments
- POST /equipment-bindings
- POST /finalize-*-confirm
- POST /notes

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- await-dispatcher-routing-decision
- await-finalize-second-confirm
- client-up-check
- client-up-decision
- contractor-assignment-decision
- on-site-work-loop
- r1-vip-internal-first
- r2-hfc-by-region
- r2-hfc-by-region-1
- r3-hfc-by-region-2
- r4-gpon-by-region
- r5-voip-anywhere
- r6-equipment-upgrade-internal
- r7-fallback-default
- wait-for-topology-capture
- wo-auto-assign-contractor
- wo-capture-equipment-bindings-at-finalize
- wo-emit-homepass-update-at-finalize

- `wo-emit-state-event`
- `wo-enforce-finalize-checklist`
- `wo-mark-pattern-b-reroute`
- `wo-tag-warranty-window`
- `wo-validate-create`

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- `WorkOrderAssigned`
- `WorkOrderAttachmentAdded`
- `WorkOrderCancelled`
- `WorkOrderClaimedByDispatcher`
- `WorkOrderClaimedByTechnician`
- `WorkOrderCreated`
- `WorkOrderEquipmentBindingsRecorded`
- `WorkOrderFinalizationPending`
- `WorkOrderFinalized`
- `WorkOrderHomePassTopologyResolved`
- `WorkOrderInProgress`
- `WorkOrderInstallationCompleted`
- `WorkOrderNoteAppended`
- `WorkOrderReassigned`
- `WorkOrderReroutedPatternB`

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.

9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 2 | **Group III:** Fulfillment & Operations | **Version:** v1.0 | **Module:** Work Order | **Satellite of:** DD_WO-01-Work_Order_Module-v1.0

The Installation flow satellite. Defines `WorkOrder_Installation_WIK.bpmn`, the Drools KJAR shape for assignment + finalization decisions, the operator-config rows seeding the framework's catalogs, the Pattern B "reassign-without-closure" mechanism for blocked installs, the equipment-binding capture at finalize, and the warranty + RPT linkage. Built on top of DD_WO-01-FRAMEWORK-v1.0; relies on its primitives without re-stating them.

Covers all Job Types in the `INSTALLATION` kind: `H01` (HFC Installation), `G07` (VOIP Installation), `DLC` (Wi-Fi Extender Install — paid), `DL4` (Free Extender Install), `RTV` (Rescheduled TV Install), `EQU` (Equipment Upgrade). Plus the `RPT` linkage when a repeat-within-warranty WO is raised.

1. What this flow is

The Installation flow turns "we owe this customer a working service" into a finalized WO. Caller — FUL-02 STEP-INSTALL for new-customer onboarding, OSR-ADD (future) for service-add to an existing subscriber, or BO UI for manual additions — supplies the customer, the HomePass, the equipment to install, and any operator metadata. The flow handles assignment, dispatch, on-site work, blocked-install rerouting, the strict "client must be up" finalization gate, equipment binding capture, network-topology capture (where applicable), and the warranty-linkage tagging. At finalization, the flow propagates the final on-site reality back to the HomePass record so the network topology held in SOPHIX stays accurate.

This BSS is deployable in any country Wananchi Group operates (today: KE, UG, TZ, SN). The shape of an install varies country-by-country because network deployment models vary:

- **In Kenya (WIK), the last mile is already in place** at the time the customer signs up. The HomePass record carries OLT port + PBO + splitter details captured by the network-planning team before any customer is sold. The install tech *confirms* this topology when they finalize the WO — or *corrects* it if reality on the ground differs from the planning record.
- **In Senegal (YASSN), the last mile is deployed during the install itself.** The HomePass record at install-time carries only the address + the upstream network anchor; the OLT port assignment, PBO box ID, splitter position, and customer drop ID are decided and captured *during* the install. The field tech uses a mobile app ("app terrain") to record these. Without this capture, downstream provisioning can't push the configuration to the right network element.
- **Other countries (WUG, WTZ, future deployments)** will fall somewhere between these two extremes. The framework + flow design supports the full spectrum without code changes.

The flow's behavior is shaped by two non-negotiable business rules from the workshops:

1. **Installations don't close until the client is up.** Unlike Support WOs, which can finalize with `NO_FAULT_FOUND` or `AREA_OUTAGE_OPEN`, an Installation WO that hasn't put the customer online stays open. If the field tech can't get the customer online (signal issue, missing infrastructure, drop not built), the WO is **passed** to maintenance or construction — same `wo_id`, new assigned team — and waits for the blocker to be cleared. Then it comes back to the install contractor to finalize. This is Pattern B (DD_WO-01-FRAMEWORK §3 reassignment-without-closure).
2. **Equipment bindings + network topology at finalize are mandatory for installs.** Equipment bindings (modem/ONT/decoder serials, MACs, ONU-IDs) are captured for every install regardless of country. Network topology capture (OLT port, PBO, splitter, drop ID) is captured by the tech where deployment timing requires it (SN); confirmed-from-existing-HomePass where the topology was pre-planned (KE). Both cases produce the same outcome at WO close: the HomePass record holds the as-installed topology.

The Installation flow is single-phase — `current_phase` stays NULL throughout. Unlike Shifting, Installation does not have sub-phases.

What this flow does NOT do

- **Doesn't define the WO module's state machine, REST API, schemas, contractor model, or Kafka primitives** — those live in DD_WO-01-FRAMEWORK. This satellite extends the framework with installation-specific Camunda BPMN, Drools rules, operator config rows, and one new REST endpoint for equipment bindings.
- **Doesn't own customer provisioning.** When the WO finalizes and emits `WorkOrderFinalized`, FUL-02 STEP-INSTALL (or OSR-ADD, or the future caller) picks up the event and triggers downstream provisioning via SUB-WF-ACTIVATE-01. The Installation flow's responsibility ends at "client is confirmed up; bindings recorded; WO closed."
- **Doesn't escalate to NOC.** That's the Support flow's Pattern A (`QCS` sub-WO). Installation uses Pattern B — same WO reassigned within the same wo_id; no new WO opened to maintenance.
- **Doesn't handle Shifting.** GSD / GSR / HSD / HSR are the Shifting flow's concern (DD_WO-01-FLOW-SHIFTING). Even though a Reconnect leg looks installation-like, it lives in the Shifting flow because of the paired-with-disconnect coordination.
- **Doesn't compute SLA.** Per memory rule on Phase 1: WO module captures SLA timestamps; FUL-OPS-SLA (Phase 2) computes.

Glossary additions (specific to this flow)

Term	Meaning
Pattern B	"Reassign without closure" — an installation WO blocked by a network/signal/infrastructure issue is reassigned to maintenance or construction without changing status. Same `wo_id` keeps moving through teams until the blocker is cleared, at which point it returns to the install contractor for finalization.
Equipment binding	Per-equipment record captured at finalization: kind (HFC modem, GPON ONT, ATA, decoder, etc.) + brand + model + serial + MAC + ONU-ID + action (`INSTALLED` / `REPLACED` / `RECOVERED` / `REUSED`). Stored in `wo_equipment_ref_stub`.
Network topology capture	Per-customer last-mile data captured (or confirmed) by the field tech at install time: OLT port, PBO box ID, splitter position + port, fiber drop ID, distance. Where the last mile is deployed during install (SN), this is the *origination* of the data; where the last mile is pre-built (KE), this is *confirmation* of the planned topology — with optional corrections when reality differs from the planning record. Captured as a structured note (`note_kind = network_topology_capture`) via the same REST API the BO UI uses.
App terrain	The Wananchi field-tech mobile application used in SN and emerging in other deployments. From the WO module's perspective, it is just another REST client of the WO API — calling the same `POST /notes`, `POST /attachments`, `POST /equipment-bindings`, and `POST /finalize-*-confirm` endpoints the BO UI uses. Its DD is owned separately by the Mobile Field Tech App module (future).
HomePass topology update at finalize	When an install WO finalizes, the flow propagates the captured (or confirmed) topology back to the HomePass record via an event consumed by RLM-CFG-01. This keeps SOPHIX's HomePass topology in sync with what's actually installed in the network — important regardless of whether the data was captured fresh (SN) or confirmed (KE).
Client up gate	The hard finalization check that an installation WO cannot close until evidence the customer is online: Easydocsis screenshot attached, signal/optical readings note captured, speed-test photo attached, final reason set to one of the success codes.

Term	Meaning
RPT linkage	When a customer raises a Support WO that's within the installation warranty window (6 months), the system creates an RPT repeat WO with `master_wo_id` pointing to the original install. Captures recurring-issue patterns and triggers TRC/TRD contractor-billing codes for warranty visits.
Warranty window	6 months from `completed_at` on the installation WO (per workshop). During this window: TRC code used when the original install contractor returns for a related issue (covered by install warranty, no separate invoicing); TRD code for warranty visits by a different contractor (different contractor billing arrangement).

2. BPMN — `WorkOrder_Installation_WIK.bpmn`

One BPMN per operator (v1.0 ships WIK only). The process key is `WorkOrder_Installation_WIK`; future operators get `WorkOrder_Installation_WUG`, `_WTZ`, `_YASSN`. Same workers run; per-operator variation is in operator-config rows (§3) + the operator's Drools KJAR (§4).

2.1 Process shape

```
          start
            ■
            ▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ wo-validate-create ■  external task — framework worker
■■■■■■■■■■■■■■■■■■■■■■■■■■■
            ■
            ▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ wo-auto-assign-contractor■  external task — framework worker (consults Drools KJAR)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
            ■
            ▼   (status → ASSIGNED)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ await-contractor-pick     ■  user task — TEAM mode: wait for /claim (internal-tech race
■  OR await-dispatcher-ack  ■              or external-dispatcher ack); TECHNICIAN mode: skip
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
            ■   (status stays ASSIGNED until claim happens)
            ▼
■■■■■■■■■■■■■■■■■■■■■■■■■■
■ await-tech-start  ■  user task — wait for /start  (status → IN_PROGRESS)
■■■■■■■■■■■■■■■■■■■■■■■■■■
            ■
            ▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ on-site-work-loop (collect notes/attachments) ■  manual; tech adds notes + attachments
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■  via REST; loops until contractor first-confirms
            ■
            ▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ await-finalize-first-confirm■  user task — wait for /finalize-first-confirm
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■     (status → FINALIZATION_PENDING)
            ■
            ▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Drools gateway: client-up-check       ■  business rule task: evaluates "is client up?"
■   (input: final_reason_code,          ■
■    optical/signal notes, attachments) ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
             ■ YES — proceed         ■ NO — must reroute
             ▼                       ▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■   ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■await-finalize-second-confirm■    ■ wo-mark-pattern-b-reroute       ■  flow worker
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■   ■ (revert to IN_PROGRESS;          ■
            ■                       ■  /reassign to new team;          ■
            ▼                       ■  emit WorkOrderReroutedPatternB)■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■       ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■wo-capture-equipment-    ■                  ■
■bindings-at-finalize     ■  flow worker     ▼  (loop back to on-site-work-loop with new team)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
            ■
            ▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■wo-enforce-finalize-checklist    ■  framework worker — verify required notes + attachments + bindings
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
            ■ pass
            ▼
```

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■wo-emit-state-event             ■  framework worker — emit WorkOrderFinalized
■ (status → COMPLETED)            ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
         ■
         ▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■wo-tag-warranty-window       ■  flow worker — set warranty_expires_at metadata
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
         ■
         ▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■wo-emit-homepass-update-at-finalize  ■  flow worker — emit WorkOrderHomePassTopologyResolved
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■     (RLM-CFG-01 updates HomePass record)
         ■
       end

     + boundary message catch (interrupting): WorkOrderCancellationRequested
       → wo-mark-cancelled (framework worker) → status=CANCELLED → end-cancelled
```

The flow has one Drools gateway (the `client-up-check` after first-confirm), several flow-specific workers (`wo-mark-pattern-b-reroute`, `wo-capture-equipment-bindings-at-finalize`, `wo-tag-warranty-window`, `wo-emit-homepass-update-at-finalize`), and a Pattern B loop that bounces a stuck install back to IN_PROGRESS with a new team.

2.2 BPMN XML — key snippets

Drools gateway (client-up-check):

```
<bpmn:businessRuleTask
    id="client-up-check"
    name="Drools: is client up?"
    camunda:resultVariable="clientUpDecision"
    camunda:decisionRef="client-up-decision"
    camunda:decisionRefBinding="latest"
    camunda:mapDecisionResult="singleEntry">
  <bpmn:extensionElements>
    <camunda:inputParameter name="woId">${woId}</camunda:inputParameter>
    <camunda:inputParameter name="jobTypeCode">${jobType}</camunda:inputParameter>
    <camunda:inputParameter name="finalReasonCode">${finalReason}</camunda:inputParameter>
  </bpmn:extensionElements>
</bpmn:businessRuleTask>

<bpmn:exclusiveGateway id="gateway-client-up">
  <bpmn:incoming>flow-from-drools</bpmn:incoming>
  <bpmn:outgoing>flow-client-up-yes</bpmn:outgoing>
  <bpmn:outgoing>flow-client-up-no</bpmn:outgoing>
</bpmn:exclusiveGateway>

<bpmn:sequenceFlow id="flow-client-up-yes" sourceRef="gateway-client-up" targetRef="await-finalize-second-confirm">
  <bpmn:conditionExpression>${clientUpDecision == 'CLIENT_UP'}</bpmn:conditionExpression>
</bpmn:sequenceFlow>

<bpmn:sequenceFlow id="flow-client-up-no" sourceRef="gateway-client-up" targetRef="wo-mark-pattern-b-reroute">
  <bpmn:conditionExpression>${clientUpDecision == 'CLIENT_NOT_UP_REROUTE'}</bpmn:conditionExpression>
</bpmn:sequenceFlow>
```

The Drools decision `client-up-decision` lives in the operator's KJAR (§4). It reads the WO's final reason code, the latest signal/optical notes, and the attachments. Returns `CLIENT_UP` (proceed to second-confirm) or `CLIENT_NOT_UP_REROUTE` (Pattern B).

Pattern B reroute service task:

```
<bpmn:serviceTask
    id="wo-mark-pattern-b-reroute"
    name="Pattern B reroute"
    camunda:type="external"
    camunda:topic="wo-mark-pattern-b-reroute"/>
```

This is a flow worker (§7) that:

1. Reverts WO status FINALIZATION_PENDING → IN_PROGRESS (framework `/finalize-revert` semantics).
2. Calls `/reassign` to the next team in the routing chain (maintenance for signal-issues; construction for missing-drop-build).
3. Emits `WorkOrderReroutedPatternB` event (§8.2).
4. Releases the BPMN to loop back to `on-site-work-loop`.

Cancellation boundary — inherited from framework §2.4. Same pattern.

2.3 Per-operator BPMN composition

The Installation flow can be composed differently per operator without code change. v1.0 KE WIK ships the shape above. Future operators can:

- Skip the `wo-auto-assign-contractor` step (WUG might require manual assignment from day 1).
- Add a pre-dispatch survey step (YASSN may require a feasibility check before sending the install tech).
- Tune the Drools `client-up-check` thresholds (different optical levels acceptable in different regions).
- Insert extra user tasks for compliance approvals.

All of these are BPMN edits, not framework or code changes. The framework provides the primitives; the BPMN composes them per operator's reality.

3. Per-operator + per-flow configuration rows

This flow seeds the following framework catalogs at v1.0 deploy time. All rows are operator-scoped to `WIK`.

3.1 `wo_flow_config` row

operator_code	kind	bpmn_process_key	drools_kjar_ref
`WIK`	`INSTALLATION`	`WorkOrder_Installation_WIK`	`wo-installation-wik-v1.0.kjar`

3.2 `wo_job_type_catalog` rows

operator_code	job_type_code	kind	display_name	description	network_type	requires_site_visit	warranty_days
`WIK`	`H01`	`INSTALLATION`	`HFC Installation`	`New install for HFC clients`	`HFC`	true	180
`WIK`	`G07`	`INSTALLATION`	`VOIP Installation`	`Install a VOIP handset`	`NULL`	true	180
`WIK`	`DLC`	`INSTALLATION`	`Wi-Fi Extender Install (paid)`	`Client paid for Wi-Fi extender — team delivers + installs`	`NULL`	true	180
`WIK`	`DL4`	`INSTALLATION`	`Free Extender Install`	`Same as DLC but free — raised by retention to track free extenders`	`NULL`	true	180
`WIK`	`RTV`	`INSTALLATION`	`Rescheduled TV Install`	`Client requests TV after the initial install`	`NULL`	true	180
`WIK`	`EQU`	`INSTALLATION`	`Equipment Upgrade`	`Replace/upgrade existing equipment (newer modem/ONT/decoder)`	`NULL`	true	0
`WIK`	`GIN`	`INSTALLATION`	`GPON Internet Installation`	`New install for GPON internet clients`	`GPON`	true	180
`WIK`	`GTV`	`INSTALLATION`	`GPON TV Installation`	`Install GPON TV decoder + service`	`GPON`	true	180

Note: warranty_days = 0 for EQU because an equipment upgrade isn't a fresh install; the original install's warranty continues to apply.

3.3 `wo_note_kind_registry` rows (installation-specific kinds)

Inherits framework's generic kinds (findings, solution, csr_note, dispatcher_note, customer_appointment, hfc_signal_readings, optical_readings, final_reason_set). Adds:

operator_code	note_kind	schema_jsonb (excerpt)	append_only
`WIK`	`pre_install_address_check`	`{"required":["addressConfirmedBy"],"properties":{"addressConfirmedBy":{"type":"string"},"comments":{"type":"string"}}}`	true
`WIK`	`pattern_b_reroute_reason`	`{"required":["reasonCode","blockerDescription"],"properties":{"reasonCode":{"enum":["SIGNAL_ISSUE","NETWORK_ISSUE","DROP_NOT_BUILT","INFRASTRUCTURE_GAP","OTHER"]}}}`	true
`WIK`	`client_up_evidence`	`{"required":["evidenceType"],"properties":{"evidenceType":{"enum":["EASYDOCSIS_SCREENSHOT","SPEED_TEST_PHOTO","OPTICAL_READING_PHOTO","CUSTOMER_SIGN_OFF"]}}}`	true
`WIK`	`equipment_intent`	`{"properties":{"plannedEquipment":{"type":"array","items":{"type":"object","properties":{"equipmentKind":{"type":"string"},"qty":{"type":"integer"}}}}}}`	false
`WIK`	`network_topology_confirmation`	`{"required":["confirmedAsPlanned"],"properties":{"confirmedAsPlanned":{"type":"boolean"},"corrections":{"type":"object","properties":{"oltPort":{"type":"string"},"pboId":{"type":"string"},"splitterPort":{"type":"integer"},"dropId":{"type":"string"}}}}}`	true
`YASSN`	`pre_install_address_check`	`{"required":["addressConfirmedBy"],"properties":{"addressConfirmedBy":{"type":"string"}}}`	true
`YASSN`	`pattern_b_reroute_reason`	`{"required":["reasonCode","blockerDescription"]}`	true
`YASSN`	`client_up_evidence`	`{"required":["evidenceType"]}`	true
`YASSN`	`network_topology_capture`	`{"required":["oltPort","pboId","splitterPort","dropId","distanceM"],"properties":{"oltPort":{"type":"string","description":"OLT slot/port e.g. 0/3/2/4"},"pboId":{"type":"string","description":"Boîtier Pied d'Immeuble (PBO) identifier"},"splitterPort":{"type":"integer","description":"1-16 splitter port"},"dropId":{"type":"string","description":"Customer fiber drop ID"},"distanceM":{"type":"integer"},"plannedVsActualNotes":{"type":"string"}}}`	true

Two distinct note kinds, on purpose:

- `network_topology_confirmation` (WIK): the HomePass record already carries the planned topology; the tech *confirms* it matches reality, or supplies corrections to specific fields. Fields are optional in the corrections object.
- `network_topology_capture` (YASSN): the HomePass record at install time has no last-mile detail; the tech *captures* the full set of fields. All five core fields (oltPort, pboId, splitterPort, dropId, distanceM) are required.

Both feed the same downstream effect (HomePass update event at finalize), but the validation contract differs because the workflow differs. Future operators register whichever variant matches their deployment model.

3.4 `wo_finalization_requirements` rows

These rows are read by the framework's `wo-enforce-finalize-checklist` worker at second-confirm. The specific-job-type row overrides the default (NULL) row.

operator_code	kind	job_type_code	required_note_kinds	required_attachment_categories	min_attachments_per_category
WIK	INSTALLATION	NULL	["findings","solution","final_reason_set","client_up_evidence","network_topology_confirmation"]	["setup_photo"]	NULL
WIK	INSTALLATION	H01	["findings","solution","hfc_signal_readings","final_reason_set","client_up_evidence","network_topology_confirmation"]	["setup_photo","speed_test","rf_optical_reading"]	NULL
WIK	INSTALLATION	GIN	["findings","solution","optical_readings","final_reason_set","client_up_evidence","network_topology_confirmation"]	["setup_photo","rf_optical_reading","speed_test"]	NULL
WIK	INSTALLATION	GTV	["findings","solution","optical_readings","final_reason_set","client_up_evidence","network_topology_confirmation"]	["setup_photo","rf_optical_reading"]	NULL
WIK	INSTALLATION	G07	["findings","solution","final_reason_set","client_up_evidence"]	["setup_photo"]	NULL
WIK	INSTALLATION	DLC	["findings","solution","final_reason_set"]	["setup_photo"]	NULL
WIK	INSTALLATION	DL4	["findings","solution","final_reason_set"]	["setup_photo"]	NULL
WIK	INSTALLATION	RTV	["findings","solution","final_reason_set"]	["setup_photo"]	NULL
WIK	INSTALLATION	EQU	["findings","solution","final_reason_set"]	["setup_photo","damaged_equipment"]	{"damaged_equipment":1}
YASSN	INSTALLATION	NULL	["findings","solution","final_reason_set","client_up_evidence","network_topology_capture"]	["setup_photo","pbo_photo"]	NULL
YASSN	INSTALLATION	GIN	["findings","solution","optical_readings","final_reason_set","client_up_evidence","network_topology_capture"]	["setup_photo","pbo_photo","rf_optical_reading","speed_test"]	NULL

Equipment bindings are an additional check enforced by the flow's own `wo-capture-equipment-bindings-at-finalize` worker (§7) — see §6 for the binding requirements per Job Type, kept separate from `wo_finalization_requirements` because they live in `wo_equipment_ref_stub` (different table).

Note on the WIK GPON requirements above. `network_topology_confirmation` is required even when the HomePass already has the topology — the tech must explicitly confirm-or-correct it. This is what closes the gap between planning data and as-installed reality. For Job Types where the topology doesn't apply (G07 VOIP — voice doesn't need fiber path detail, DLC/DL4 extenders, RTV TV-only, EQU equipment upgrade — the network is already up), the confirmation note isn't required.

Note on the YASSN requirements. The `network_topology_capture` note kind is required for every GPON-based install — without those five fields no provisioning can happen. `pbo_photo` (a category for the photo of the PBO box where the fiber was spliced) is required as physical evidence of the splice work.

3.5 `wo_final_reason_catalog` rows

operator_code	kind	job_type_code	final_reason_code	display_name	triggers_escalation_event
`WIK`	`INSTALLATION`	NULL	`INSTALL_COMPLETED_CUSTOMER_UP`	`Install completed, customer up`	false
`WIK`	`INSTALLATION`	NULL	`INSTALL_COMPLETED_TV_DELIVERED`	`Install completed, TV channels delivered`	false
`WIK`	`INSTALLATION`	NULL	`INSTALL_COMPLETED_VOIP_DELIVERED`	`Install completed, VOIP service active`	false
`WIK`	`INSTALLATION`	NULL	`EQUIPMENT_UPGRADED_OK`	`Equipment upgraded successfully`	false
`WIK`	`INSTALLATION`	NULL	`CUSTOMER_NOT_AVAILABLE`	`Customer not available — reschedule`	false
`WIK`	`INSTALLATION`	NULL	`CUSTOMER_REFUSED_INSTALL`	`Customer refused install at last moment`	false
`WIK`	`INSTALLATION`	NULL	`ADDRESS_NOT_FOUND`	`Address could not be located`	false

Note: there is intentionally NO `COULD_NOT_RESOLVE_ESCALATED` final reason for INSTALLATION. Installations don't escalate (Pattern A is Support-only); installations reroute (Pattern B). A failed-to-bring-up install does not finalize — it reroutes and stays open. Therefore `triggers_escalation_event` is `false` for every install final reason.

3.6 Operator variation surface

Six axes vary per operator without code change. New operators are onboarded by editing these axes, never by writing new flow code:

Axis	Where it lives	Variation example
BPMN composition	`WorkOrder_Installation_<OP>.bpmn` deployed to Camunda	WIK: auto-assign → user task pick → on-site → 2-step finalize. YASSN: same, plus a `wait-for-topology-capture` user task between on-site work and first-confirm. WUG (future): may insert a feasibility-survey user task before assignment.
Drools KJAR	`wo-installation-<op>-v<n>.kjar` deployed to Drools	WIK: routing rules listed in §4.1 + client-up thresholds in §4.2. YASSN: different routing (regional contractors), different optical thresholds, additional rule that requires `network_topology_capture` note before allowing finalization.
`wo_job_type_catalog` **rows**	Operator-scoped catalog	WIK: H01, G07, GIN, GTV, DLC, DL4, RTV, EQU. YASSN: a different code set reflecting the SN service lines. Codes do not need to be globally unique — they are scoped per operator.
`wo_note_kind_registry` **rows**	Operator-scoped registry with per-kind JSONB schemas	WIK: `network_topology_confirmation` (confirmation against pre-planned data). YASSN: `network_topology_capture` (fresh capture). Both feed the same downstream HomePass-update mechanism but enforce different validation.
`wo_finalization_requirements` **rows**	Operator-scoped, job-type-scoped	WIK: required notes include `client_up_evidence` + `network_topology_confirmation`. YASSN: required notes include `network_topology_capture` and additional `pbo_photo` attachment category.
`wo_final_reason_catalog` **rows**	Operator-scoped, kind-scoped, job-type-scoped	WIK: success codes listed in §3.5. Future operators add their own.

The framework provides primitives; the flow's logic is parameterized by these six axes; downstream consumers (RLM-CFG-01 for HomePass updates, FUL-02 for activation, Finance for invoicing) consume the same event types regardless of operator.

Why this matters. A single Installation flow design can serve countries with fundamentally different deployment models. KE (last-mile pre-built; topology confirmed at install) and SN (last-mile deployed during install; topology captured fresh) use the same BPMN shape, the same workers, the same REST API, the same Kafka events — they differ only in the configuration. New countries with hybrid models (some areas pre-built, others built-during-install) can layer their own variation without forking code.

4. Drools KJAR — `wo-installation-wik-v1.0.kjar`

The flow's KJAR holds two named decisions, both invoked from the BPMN via `businessRuleTask` nodes:

1. `contractor-assignment-decision` — invoked by the framework's `wo-auto-assign-contractor` worker. Picks contractor + team + (optionally) technician for the WO.
2. `client-up-decision` — invoked at the post-first-confirm gateway. Decides whether the WO can proceed to second-confirm or must reroute (Pattern B).

4.1 `contractor-assignment-decision`

Inputs: `{operatorCode, kind, jobTypeCode, networkType, techRegionCode, customerVipTier?, scheduledAt}`.

Output: `ContractorAssignmentDecision { contractorId, teamId, technicianId?, mode, considered[] }`.

v1.0 KE WIK rules (one DRL file, ordered by salience):

Rule name	Trigger condition	Decision
`r1-vip-internal-first`	`customerVipTier == "VIP"`	Prefer `ctr_Z071H` (internal in-house support team) regardless of network type. Mode=TECHNICIAN if a tech is available; else TEAM.
`r2-hfc-by-region`	`networkType == "HFC"` && tech region in `["NRB-KAREN","NRB-LANGATA","NRB-LAVINGTON"]`	`ctr_Z071H` TEAM mode.
`r3-hfc-by-region-2`	`networkType == "HFC"` && tech region in `["NRB-WESTLANDS","NRB-PARKLANDS"]`	`ctr_GTECH` TEAM mode.
`r4-gpon-by-region`	`networkType == "GPON"` && tech region in `["NRB-KAREN","NRB-LANGATA"]`	`ctr_GTECH` TEAM mode.
`r5-voip-anywhere`	`jobTypeCode == "G07"`	`ctr_Z071H` TEAM mode (internal team handles VOIP).
`r6-equipment-upgrade-internal`	`jobTypeCode == "EQU"`	`ctr_Z071H` TEAM mode.
`r7-fallback-default`	Otherwise	`ctr_ADRIAN` TEAM mode (external — handles whatever doesn't match above).

Considered alternatives are recorded for audit. The Drools KJAR file lives at `/sophix/drools/wo-installation-wik-v1.0.kjar` and is deployed alongside the BPMN. Operator-specific overrides happen by deploying a different KJAR per operator (`wo-installation-wug-v1.0.kjar`, etc.); the rules can differ entirely per operator without code change.

Sample Drools rule (illustrative):

```
package com.wananchi.sophix.wo.installation.wik;

import com.wananchi.sophix.wo.dto.AssignmentInput;
import com.wananchi.sophix.wo.dto.AssignmentDecision;

global java.util.List considered;

rule "r2-hfc-by-region-1"
    salience 100
    when
        $i : AssignmentInput(
            operatorCode == "WIK",
            networkType == "HFC",
            techRegionCode in ("NRB-KAREN","NRB-LANGATA","NRB-LAVINGTON")
        )
```

```
        then
            AssignmentDecision d = new AssignmentDecision();
            d.setContractorId("ctr_Z071H");
            d.setTeamId("team_rapid_response");
            d.setMode("TEAM");
            d.setReason("HFC + region match, internal in-house team");
            insert(d);
            considered.add("ctr_Z071H");
    end
```

4.2 `client-up-decision`

Inputs: {`woId, jobTypeCode, finalReasonCode, latestSignalReading?, latestOpticalReading?, attachmentCategories[]`}.

Output: "`CLIENT_UP`" or "`CLIENT_NOT_UP_REROUTE`".

v1.0 KE WIK rules:

Rule	Decision
Final reason in {`INSTALL_COMPLETED_CUSTOMER_UP, INSTALL_COMPLETED_TV_DELIVERED, INSTALL_COMPLETED_VOIP_DELIVERED, EQUIPMENT_UPGRADED_OK`} AND required client-up-evidence note present AND for HFC: latest `hfc_signal_readings` shows `downstreamLevel` ∈ `[-15, 15]` dBm AND `snr ≥ 30` dB AND for GPON: latest `optical_readings` shows `ontRxDbm` ∈ `[-28, -8]` dBm	`CLIENT_UP`
Final reason in {`CUSTOMER_NOT_AVAILABLE, CUSTOMER_REFUSED_INSTALL, ADDRESS_NOT_FOUND`}	`CLIENT_UP` (these are administratively-finalized — the install didn't happen for non-network reasons; flow closes normally, no Pattern B reroute)
Final reason in success codes BUT signal/optical thresholds NOT met	`CLIENT_NOT_UP_REROUTE`
Final reason in success codes BUT required client_up_evidence note missing	Framework `wo-enforce-finalize-checklist` worker already rejects this at second-confirm; Drools doesn't need to handle it. The gateway only runs after first-confirm.

The signal/optical thresholds come from the workshop and from the Easydocsis tool's own published ranges. Operator override possible by editing the KJAR.

5. Pattern B reassignment — narrative

The Installation flow's defining behavior is Pattern B: an install WO that can't put the customer online stays open. Same `wo_id`, new team, no finalize. The workshop (escalation doc):

Installation WOs are NOT finalised if the client is not up. If a signal/network issue prevents the install, the same installation WO is passed to the relevant team (maintenance, construction) — not closed and reopened. The WO must come back to the install contractor to be finalised once the blocker is resolved.

How the flow implements this:

5.1 Trigger

After first-confirm finalize, the BPMN runs the `client-up-decision` Drools gateway. If the decision is `CLIENT_NOT_UP_REROUTE`, the flow takes the no-branch.

5.2 Reroute mechanics

The flow worker `wo-mark-pattern-b-reroute` (§7) executes these steps in one transaction:

1. **Revert status.** `FINALIZATION_PENDING` → `IN_PROGRESS`. Framework's `/finalize-revert` semantics. `final_reason_code` cleared. `finalization_pending_at` cleared.
2. **Append the reroute note.** Write a structured `wo_notes` row with `note_kind = pattern_b_reroute_reason` containing the blocker. Example: `{"reasonCode":"SIGNAL_ISSUE","blockerDescription":"ONT RX power -32 dBm, below acceptable threshold -28 dBm. Need optical line check by maintenance."}`.
3. **Determine the next team.** Based on the reason:

- `SIGNAL_ISSUE` / `NETWORK_ISSUE` → `ctr_NOC_MAINT` / `team_noc_maintenance` (internal NOC team handles signal investigation).

- `DROP_NOT_BUILT` / `INFRASTRUCTURE_GAP` → `ctr_CONSTRUCTION` / `team_construction` (internal construction team; if not configured for the operator yet, falls back to NOC).

- `OTHER` → routed per `dispatcher_note` decision — flow worker pauses and emits a user task `await-dispatcher-routing-decision` for human selection.

4. **Reassign via framework `/reassign` API.** Carries `assignment_reason = "escalation_pattern_b"`. Status stays IN_PROGRESS (per framework R-WO-FW-7). `wo_assignment_history` row written; `WorkOrderReassigned` emitted.

5. **Emit** `WorkOrderReroutedPatternB` (flow-specific event, §8.2). Carries the blocker reason + original install contractor (so when the WO eventually returns to that contractor for finalize, the audit chain is clear).

6. **Loop the BPMN.** Execution returns to `on-site-work-loop` — the new team picks up the WO, does their work (e.g., NOC verifies optics remotely; maintenance dispatches a tech to fix the line), adds notes/attachments, and either:

- **Returns the WO to the install contractor.** A subsequent `/reassign` back to the original install contractor (`assignment_reason = "return_to_originating_contractor"`). The new dispatcher_note explains what was fixed. Install contractor re-attempts the install — adds new findings/solution/signal readings — and first-confirm-finalizes again.

- **Loops to another team** if the first reroute didn't fully clear the blocker (NOC's diagnosis says construction needed, so maintenance reassigns to construction).

5.3 No-cycle invariant

Pattern B can in principle loop indefinitely (NOC → maintenance → NOC → install → …). The framework doesn't constrain this; the dispatcher's BO UI surfaces a "Pattern B loop count" widget on each WO showing how many reroutes have occurred so dispatchers can intervene when a WO loops too many times. v1.0 KE workshop is comfortable leaving this as a process discipline rather than a hard rule (workshop quote: *"the install contractor takes ownership end-to-end; if it loops, that's a real-world problem the dispatch team will see"*).

A future enhancement could add a configurable cap (`max_pattern_b_loops = 5` → escalation to NOC operations lead). v1.0 ships without the cap.

5.4 Returning to install contractor — invariant

When the WO is finally returned to the install contractor and they do the final attempt:

- Status stays IN_PROGRESS throughout the loop. No COMPLETED status transition until second-confirm fires.

- `assigned_at` stays at the original first-assignment timestamp; framework's R-WO-FW-19 says `*_at` columns set exactly once. The reassignments only update `last_updated_at`.

- `wo_assignment_history` retains the full chain: original install contractor → NOC → (maintenance) → install contractor. Useful for SLA analysis (Phase 2) and for Finance to invoice each team for their portion of the work.

6. Equipment bindings at finalize

Per the workshop, an installation WO finalization records exactly what was physically installed. The flow worker `wo-capture-equipment-bindings-at-finalize` runs in the BPMN between Drools gateway-pass and second-confirm.

6.1 Required bindings per Job Type

Job Type	Minimum required bindings	Notes
`H01` (HFC Install)	≥1 with `equipment_kind = HFC_MODEM`, action=`INSTALLED`	Mandatory serial + MAC
`G07` (VOIP Install)	≥1 with `equipment_kind = ATA`, action=`INSTALLED`	Mandatory serial
`GIN` (GPON Internet)	≥1 with `equipment_kind = GPON_ONT`, action=`INSTALLED`	Mandatory serial + MAC + ONU-ID
`GTV` (GPON TV)	≥1 with `equipment_kind = TV_DECODER`, action=`INSTALLED` + optionally GPON_ONT if not already present from a prior install	Mandatory serial
`DLC`, `DL4` (Extender)	≥1 with `equipment_kind = WIFI_EXTENDER`, action=`INSTALLED`	Mandatory serial

Job Type	Minimum required bindings	Notes
RTV (Rescheduled TV)	≥1 with `equipment_kind = TV_DECODER`, action=`INSTALLED`	Mandatory serial
EQU (Equipment Upgrade)	≥2: one with action=`REPLACED` (old serial in `replaced_serial`) AND one with action=`INSTALLED` (new equipment)	Both serials required

6.2 Capture mechanics

The field tech (or contractor's dispatcher acting on their behalf) records bindings via the new endpoint:

```
POST /api/work-orders/{woId}/equipment-bindings HTTP/1.1
Content-Type: application/json

{
  "bindings": [
    {
      "equipmentKind":   "HFC_MODEM",
      "brand":           "Hitron",
      "model":           "CGNV4",
      "serialNumber":    "HFC-MODEM-XYZ123",
      "macAddress":      "00:1A:2B:3C:4D:5E",
      "action":          "INSTALLED",
      "notes":           "Mounted on living-room wall outlet"
    }
  ],
  "recordedBy":          "tech_01HZ_KAREN_03",
  "recordedByRole":      "FIELD_TECH"
}
```

Framework persists rows to `wo_equipment_ref_stub`, emits `WorkOrderEquipmentBindingsRecorded` event (§8.2). The endpoint can be called multiple times during the on-site-work-loop; each call appends bindings.

6.3 Validation at second-confirm

The flow worker `wo-capture-equipment-bindings-at-finalize` reads the WO's `wo_equipment_ref_stub` rows, looks up the Job Type's required-bindings rule from §6.1, and verifies the minimum constraints. If missing → 400 `EQUIPMENT_BINDINGS_INCOMPLETE` with the list of missing kinds; WO stays in FINALIZATION_PENDING; tech captures the missing bindings; retries second-confirm.

Bindings recorded via this endpoint also flow into the `WorkOrderFinalized` event's `equipmentBindings[]` array (extending the framework's payload structure — flow can extend, never replace, framework events).

7. Flow-specific workers

Three workers live in this flow on top of the framework's generic ones:

7.1 `wo-mark-pattern-b-reroute`

External task topic: `wo-mark-pattern-b-reroute`.

Input: process variables `{woId, finalReasonCode, blockerReasonCode, blockerDescription}`.

Action:

1. Revert WO via framework `/finalize-revert` (internal call).
2. Append `pattern_b_reroute_reason` note.
3. Compute next team from `blockerReasonCode` lookup table (§5.2).
4. Reassign via framework `/reassign`.
5. Emit `WorkOrderReroutedPatternB` Kafka event.

Lock duration: 10s. Max retries: 3 (each retry preserves idempotency via the underlying framework APIs).

7.2 `wo-capture-equipment-bindings-at-finalize`

External task topic: `wo-capture-equipment-bindings-at-finalize`.

Input: process variables `{woId, jobTypeCode}`.

Action:

1. Query `wo_equipment_ref_stub` for the WO.
2. Apply the per-Job-Type rule from §6.1.
3. If pass: emit `WorkOrderEquipmentBindingsRecorded` summary event, complete the Camunda task with `bindingsValidated=true`.
4. If miss: complete the task with `bindingsValidated=false, missingKinds=[...]`. The BPMN gateway after this task routes back to `on-site-work-loop` for the contractor to add the missing bindings.

Lock duration: 5s.

7.3 `wo-tag-warranty-window`

External task topic: `wo-tag-warranty-window`.

Runs after `wo-emit-state-event` confirms `WorkOrderFinalized`. Action:

1. Read `wo_job_type_catalog` for the WO's `job_type_code`, get `warranty_days`.
2. Compute `warranty_expires_at = completed_at + warranty_days`.
3. UPDATE the WO's `metadata` JSONB to include `{"warrantyExpiresAt": "<iso>", "warrantyDays": <int>}`.
4. Complete the Camunda task.

Lock duration: 3s.

Future repeat-WO consumers (Support flow's RPT linkage decision) read `metadata.warrantyExpiresAt` to determine whether to spawn an RPT WO with `master_wo_id = <this install wo>`.

7.4 `wo-emit-homepass-update-at-finalize`

External task topic: `wo-emit-homepass-update-at-finalize`.

Runs in the BPMN between `wo-tag-warranty-window` and the end event. Action:

1. Read the WO's network-topology note(s). The flow looks for either `network_topology_capture` (if the operator's note registry has it) or `network_topology_confirmation` (otherwise). At most one of the two is registered per operator.
2. Resolve the topology values:

- For `network_topology_capture`: the captured fields are used as-is.
- For `network_topology_confirmation`: if `confirmedAsPlanned=true`, the existing HomePass topology is treated as canonical. If `confirmedAsPlanned=false`, the `corrections` object's fields override the corresponding HomePass fields.

3. Emit `WorkOrderHomePassTopologyResolved` event (§8 below) carrying the resolved topology. RLM-CFG-01 consumes this event and updates the HomePass record.
4. Complete the Camunda task.

If neither note kind is registered for the operator (e.g., a kind where topology has no meaning — VOIP-only flows where there's no fiber), the worker no-ops and completes without emitting.

Lock duration: 5s.

The framework also accepts a corollary endpoint `POST /api/work-orders/{woId}/topology-replay` that re-emits the event — used in operations recovery if RLM-CFG-01 missed the original event due to a downstream outage. Documented in §9.2.

8. Flow-specific events

This flow extends the framework's event catalog (DD_WO-01-FRAMEWORK §7) with three new events. All on the `sophix.work-orders.*` topic family.

8.1 Event table

Event type	Topic	Emitted when	Key consumers
`WorkOrderReroutedPatternB`	`sophix.work-orders.rerouted-pattern-b`	Pattern B reassignment triggered after `client-up-decision` says `CLIENT_NOT_UP_REROUTE`	NOC dashboards, dispatcher BO UI, audit, SLA tracker
`WorkOrderEquipmentBindingsRecorded`	`sophix.work-orders.equipment-bindings-recorded`	`/equipment-bindings` endpoint called OR `wo-capture-equipment-bindings-at-finalize` worker validates bindings at finalize	Future Inventory module, Finance (contractor invoicing per-equipment), audit
`WorkOrderHomePassTopologyResolved`	`sophix.work-orders.homepass-topology-resolved`	Install finalization — topology captured (or confirmed) is now authoritative for the HomePass	**RLM-CFG-01** (updates HomePass record), audit, NOC topology dashboards
`WorkOrderInstallationCompleted`	`sophix.work-orders.installation-completed`	After `WorkOrderFinalized` for an INSTALLATION-kind WO, with equipment bindings + warranty window + resolved topology populated	**FUL-02 STEP-INSTALL** (already consumes `WorkOrderFinalized`; can choose this richer event instead), Finance, future Inventory, future RPT-linkage decision logic

Existing framework events that still fire on the install flow: `WorkOrderCreated`, `WorkOrderAssigned`, `WorkOrderClaimedByTechnician` / `WorkOrderClaimedByDispatcher`, `WorkOrderInProgress`, `WorkOrderReassigned` (fires for Pattern B reroutes too — Pattern-B is a kind of reassignment), `WorkOrderNoteAppended`, `WorkOrderAttachmentAdded`, `WorkOrderFinalizationPending`, `WorkOrderFinalized`, `WorkOrderCancelled`.

8.2 Sample event — `WorkOrderReroutedPatternB`


```
{
  "eventType":   "WorkOrderReroutedPatternB",
  "aggregateId": "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp":   "2026-05-18T15:30:00.512Z",
  "payload": {
    "woId":                         "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "operatorCode":                 "WIK",
    "jobTypeCode":                  "H01",
    "blockerReasonCode":            "SIGNAL_ISSUE",
    "blockerDescription":           "Downstream level -22 dBm, below acceptable threshold -15 dBm. Need RF investigation
    "previousContractorId":         "ctr_Z071H",
    "previousTeamId":               "team_rapid_response",
    "previousTechnicianId":         "tech_01HZ_KAREN_03",
    "newContractorId":              "ctr_NOC_MAINT",
    "newTeamId":                    "team_noc_maintenance",
    "newTechnicianId":              null,
    "originalInstallContractorId":  "ctr_Z071H",
    "loopCount":                    1,
    "reroutedBy":                   "wo-mark-pattern-b-reroute-worker-1",
    "reroutedAt":                   "2026-05-18T15:30:00.512Z"
  }
}
```


`originalInstallContractorId` is preserved across multiple Pattern B loops so the dispatcher always knows where to send the WO back to for the final attempt.

8.3 Sample event — `WorkOrderInstallationCompleted`


```
{
  "eventType":   "WorkOrderInstallationCompleted",
  "aggregateId": "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp":   "2026-05-18T17:45:00.123Z",
  "payload": {
    "woId":               "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "operatorCode":       "WIK",
    "kind":               "INSTALLATION",
    "jobTypeCode":        "H01",
    "networkType":        "HFC",
    "customerId":         "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "homepassId":         "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
    "originatingContextType": "ORDER",
    "originatingContextRef":  "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "completedAt":        "2026-05-18T17:45:00.123Z",
    "warrantyExpiresAt":  "2026-11-14T17:45:00.123Z",
    "warrantyDays":       180,
    "patternBLoopCount":  1,
    "finalReasonCode":    "INSTALL_COMPLETED_CUSTOMER_UP",
    "finalAssignedContractorId":  "ctr_Z071H",
    "finalAssignedTeamId":        "team_rapid_response",
    "finalAssignedTechnicianId":  "tech_01HZ_KAREN_03",
    "equipmentBindings": [
```

```
    {
      "equipmentRefId":  "eqref_01HZ_A1",
      "equipmentKind":   "HFC_MODEM",
      "brand":           "Hitron",
      "model":           "CGNV4",
      "serialNumber":    "HFC-MODEM-XYZ123",
      "macAddress":      "00:1A:2B:3C:4D:5E",
      "action":          "INSTALLED"
    }
  ],
  "topologyResolution": {
    "mode":           "CONFIRMED",
    "confirmedAsPlanned": true,
    "corrections":    null
  }
 }
}
```

For YASSN-flavored installs where the topology was captured fresh, `topologyResolution` looks like:

```
"topologyResolution": {
  "mode":           "CAPTURED",
  "oltPort":        "0/3/2/4",
  "pboId":          "PBO-DKR-MAR-127",
  "splitterPort":   8,
  "dropId":         "DROP-887234",
  "distanceM":      1450,
  "confirmedAsPlanned": null,
  "corrections":    null
}
```

8.4 Sample event — `WorkOrderHomePassTopologyResolved`

Emitted by `wo-emit-homepass-update-at-finalize` (§7.4) just before the BPMN end. Consumed by RLM-CFG-01 to update the HomePass record.

```
{
  "eventType":   "WorkOrderHomePassTopologyResolved",
  "aggregateId": "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp":   "2026-05-18T17:44:55.512Z",
  "payload": {
    "woId":         "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "operatorCode": "YASSN",
    "homepassId":   "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
    "customerId":   "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "topologyMode": "CAPTURED",
    "topology": {
      "oltPort":      "0/3/2/4",
      "pboId":        "PBO-DKR-MAR-127",
      "splitterPort": 8,
      "dropId":       "DROP-887234",
      "distanceM":    1450
    },
    "capturedAt":     "2026-05-18T15:22:00.000Z",
    "capturedBy":     "tech_01HZ_DKR_07",
    "capturedByRole": "FIELD_TECH",
    "noteId":         "note_01HZ_077"
  }
}
```

For a WIK-style confirmation case:

```
{
  "eventType":   "WorkOrderHomePassTopologyResolved",
  "payload": {
    "operatorCode": "WIK",
    "topologyMode": "CONFIRMED_AS_PLANNED",
    "homepassId":   "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
    ...
  }
}
```

Or when the WIK tech finds reality differs from planning:

```
{
  "eventType":   "WorkOrderHomePassTopologyResolved",
  "payload": {
    "operatorCode": "WIK",
    "topologyMode": "CONFIRMED_WITH_CORRECTIONS",
    "homepassId":   "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
    "corrections": {
      "splitterPort": 11
    },
```

```
    ...
  }
}
```

RLM-CFG-01's consumer logic:

- `CAPTURED` → write the full topology fields onto the HomePass record (previously NULL or partially populated).
- `CONFIRMED_AS_PLANNED` → no HomePass change needed; optional audit log entry "topology confirmed at install".
- `CONFIRMED_WITH_CORRECTIONS` → overwrite only the fields listed in `corrections`.

9. REST API additions

This flow adds one endpoint on top of the framework's REST surface (DD_WO-01-FRAMEWORK §8). The endpoint is specific to installation-kind WOs but lives at the same `/api/work-orders/{woId}/...` path because it operates on a WO.

9.1 `POST /api/work-orders/{woId}/equipment-bindings` — Record equipment bindings

Body (see §6.2 for full example):

```
{
  "bindings": [
    {
      "equipmentKind":   "HFC_MODEM",
      "brand":           "Hitron",
      "model":           "CGNV4",
      "serialNumber":    "HFC-MODEM-XYZ123",
      "macAddress":      "00:1A:2B:3C:4D:5E",
      "action":          "INSTALLED",
      "replacedSerial":  null,
      "notes":           "Mounted on living-room wall outlet"
    }
  ],
  "recordedBy":     "tech_01HZ_KAREN_03",
  "recordedByRole": "FIELD_TECH"
}
```

Framework persists rows to `wo_equipment_ref_stub`, audit-logs the action, emits `WorkOrderEquipmentBindingsRecorded`. Valid in any non-terminal WO status.

Errors:

Status	When
400 `INVALID_EQUIPMENT_KIND`	An entry's `equipmentKind` not in the catalog (HFC_MODEM, GPON_ONT, ATA, TV_DECODER, WIFI_EXTENDER, CABLE_DROP, SPLICE_CLOSURE)
400 `MISSING_SERIAL_FOR_KIND`	Some kinds (HFC_MODEM, GPON_ONT, ATA, WIFI_EXTENDER, TV_DECODER) require `serialNumber`; if NULL, reject
400 `REPLACED_SERIAL_REQUIRED`	Entry with `action=REPLACED` but no `replacedSerial`
404 `WO_NOT_FOUND`	wo_id doesn't exist

The endpoint is idempotent on the (woId, serialNumber, action) triple — re-submitting the same binding returns 200 with the existing `equipment_ref_id`.

9.2 `POST /api/work-orders/{woId}/topology-replay` — Re-emit HomePass topology event

Operations recovery endpoint. If RLM-CFG-01's `WorkOrderHomePassTopologyResolved` consumer missed the original event (downstream outage, dead-letter queue, manual rollback), an operations engineer can replay the event via:

```
POST /api/work-orders/{woId}/topology-replay HTTP/1.1
{
  "replayReason": "RLM-CFG-01 consumer lag — DLQ items being recovered",
  "replayedBy":   "ops_engineer_03"
}
```

Framework: verifies WO is in COMPLETED status; re-reads the topology note(s); re-emits `WorkOrderHomePassTopologyResolved` with the same payload (idempotent on the consumer side by `woId`). Audit-logged.

10. Workshop sources

The Wananchi KE Installation workflow (workshop 2026-04-29 + Bildad Gitonga transcript 2026-04-30) requires:

- A single WO per install, kind=INSTALLATION, with Job Types covering HFC (H01), GPON (GIN, GTV), VOIP (G07), TV reschedule (RTV), Wi-Fi extenders (DLC, DL4), and Equipment Upgrade (EQU).
- Assignment via the dispatcher's slot pattern: contractor picked (auto or manual), team picked (auto or manual), individual technician left to contractor's own dispatcher (team mode) for internal teams, or to the contractor's external dispatcher for external contractor teams.
- The required-before-finalize checklist (findings, solution, signal/optical readings, final reason, photos, network topology confirmation).
- Easydocsis verification: HFC and GPON installs both require the client-up evidence (Easydocsis screenshot showing the modem online, signal/optical readings within thresholds) attached to the WO before finalization.
- 2-step finalization confirm: first confirm saves, second confirm closes (with checklist).
- **Installations don't close until the client is up** — Pattern B reroute to maintenance/construction without closing the WO. Same `wo_id` returns to the install contractor for final close.
- Equipment bindings captured at finalize for contractor invoicing, future inventory reconciliation, and warranty lookup.
- **Network topology confirmation at finalize** — the tech confirms the pre-planned OLT port + PBO + splitter + drop topology held in the HomePass record, with the option to correct any field that reality differs from. HomePass record is updated with corrections at WO close.
- 6-month installation warranty. Repeat WO within warranty → RPT raised with linkage to the original install WO; TRC/TRD billing codes for contractor warranty visits.

Cross-country variation. The Senegal (YASSN) deployment captures the full topology fresh at install time (no pre-planning data), via the field-tech mobile app "app terrain" hitting the same WO REST API. This variation is supported by per-operator note-kind registration + per-operator finalization requirements (see §3.6); no code change between KE and SN models.

11. Cross-DD coordination

11.1 Module owner

This satellite is owned by the Fulfillment & Operations team alongside its parent DD (WO module). It depends on DD_WO-01-FRAMEWORK-v1.0 for all primitives.

11.2 Impacted modules

Module	Direction	Contract
WO-01-FRAMEWORK	this flow ← framework	Consumes all framework primitives: state machine, REST API, Camunda integration, schemas, generic workers, Kafka events.
FUL-02 STEP-INSTALL	FUL-02 → this flow	FUL-02 calls `POST /api/work-orders` with `kind=INSTALLATION`. Consumes `WorkOrderFinalized` + `WorkOrderInstallationCompleted` (richer event) + `WorkOrderCancelled`. Retires `install_wo_stub` from FUL-02 FRAMEWORK §6.2.
Future OSR-ADD	OSR-ADD → this flow	Service-add operations (Wi-Fi extender, TV add to existing GPON customer) create install WOs with `kind=INSTALLATION` and `originatingContextType=OSR`.
BO UI (Dispatcher Console)	BO UI → this flow	Manual WO creation, manual reassignment for Pattern B routing decisions, dispatcher acks for external teams.
Future Ticketing module	Ticketing → this flow	A customer support ticket that turns out to be a missing/late install can spawn an install WO via the framework REST API.

Module	Direction	Contract
WO-01-FLOW-SUPPORT (future Phase B sibling)	Support flow → this flow	Support flow's RPT repeat-WO decision reads `metadata.warrantyExpiresAt` set by this flow's `wo-tag-warranty-window` worker. If a Support WO is created within the warranty window of a prior install, the Support flow spawns an RPT WO with `master_wo_id` pointing to this install.
Future Inventory / Equipment Catalog	this flow → Inventory	`WorkOrderEquipmentBindingsRecorded` event consumed by Inventory once it ships. Retires `wo_equipment_ref_stub`.
RLM-CFG-01 (HomePass)	this flow ←→ RLM	Reads `homepass.tech_region_code` for assignment Drools input AND `homepass.<topology fields>` at install time (for confirmation flows). At finalize: consumes `WorkOrderHomePassTopologyResolved` to either write the captured topology to the HomePass record (CAPTURED mode), no-op (CONFIRMED_AS_PLANNED), or overwrite only the corrected fields (CONFIRMED_WITH_CORRECTIONS).
Finance	this flow → Finance	`WorkOrderInstallationCompleted` event used for contractor invoicing per-visit + per-equipment-installed.

11.3 Boundary clarifications

What's in scope for this satellite vs what's not:

- ■ Installation BPMN, Drools KJAR shape, operator-config seeds, Pattern B reroute mechanism, equipment binding capture, warranty tagging.
- ■ The WO module's state machine, REST API surface, schemas, Camunda primitives — owned by DD_WO-01-FRAMEWORK.
- ■ Support flow (kind=SUPPORT) — owned by future DD_WO-01-FLOW-SUPPORT.
- ■ Shifting flow (kind=SHIFTING) — owned by future DD_WO-01-FLOW-SHIFTING.
- ■ QCS escalation / Pattern A — owned by Support flow + future Ticketing module.
- ■ RPT WO creation — owned by Support flow (Support flow detects the warranty window and creates the RPT WO; this flow only tags the warranty window during install finalize).
- ■ Native SLA computation — Phase 2 / future FUL-OPS-SLA module.
- ■ Field Audit feature — explicitly NOT used per workshop.

11.4 Dependencies (build order)

1. **DD_WO-01-FRAMEWORK-v1.0** must be shipped before this satellite.
2. **RLM-CFG-01 (HomePass)** must exist for tech-region lookup.
3. **Drools engine + KJAR deployment** infrastructure.
4. **FUL-02 STEP-INSTALL** is the immediate consumer once this satellite ships — flip its `useStub=true` config to `useStub=false` to switch to the real WO module.
5. Future inventory / RPT consumers (Phase B successors).

11.5 Migration approach

When this flow ships alongside the framework:

1. **DDL** — the framework's tables are the only schema; this flow adds no new tables (uses `wo_equipment_ref_stub` from framework §9.3). Seed `wo_flow_config`, `wo_job_type_catalog`, `wo_note_kind_registry`, `wo_finalization_requirements`, `wo_final_reason_catalog` rows per §3.
2. **BPMN deployment** — deploy `WorkOrder_Installation_WIK.bpmn` to Camunda.
3. **Drools KJAR deployment** — deploy `wo-installation-wik-v1.0.kjar`.
4. **FUL-02 cutover** — flip config flag.

5. **Future operator onboarding (WUG/WTZ/YASSN)** — add operator-suffixed BPMN + KJAR; add per-operator config rows; no code change.

12. Runtime walkthrough — narrative

Two scenarios: a happy-path HFC install that goes through on first attempt, and a GPON install that hits a signal issue and reroutes via Pattern B before completing.

12.1 Happy path — HFC install (job_type=H01)

Scenario. Customer Jane Mwangi paid for her new HFC service yesterday. FUL-02 STEP-INSTALL just consumed `OrderPaymentReceived`. It now creates the install WO.

Step 1 — FUL-02 STEP-INSTALL calls `POST /api/work-orders`.

```
POST /api/work-orders HTTP/1.1
{
  "operatorCode":           "WIK",
  "kind":                   "INSTALLATION",
  "jobTypeCode":            "H01",
  "customerId":             "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
  "homepassId":             "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
  "originatingContextType": "ORDER",
  "originatingContextRef":  "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "idempotencyKey":         "ful-02-step-install-ord_01HZ...A",
  "metadata":               {"customerName":"Jane Mwangi","contactNumber":"+254712345678","techRegionCode":"NRB-KAREN"}
}
```

Framework creates `work_order` row (status=PENDING), starts `WorkOrder_Installation_WIK` Camunda process instance with `businessKey=wo_01HZ...Z1`, returns 201 with `woId=wo_01HZ...Z1`. Emits `WorkOrderCreated`.

Step 2 — BPMN runs `wo-validate-create` then `wo-auto-assign-contractor`. The latter invokes the Drools `contractor-assignment-decision`. With inputs `{operatorCode=WIK, kind=INSTALLATION, jobTypeCode=H01, networkType=HFC, techRegionCode=NRB-KAREN}`, rule `r2-hfc-by-region-1` fires: pick `ctr_Z071H` + `team_rapid_response`, TEAM mode. Framework: updates row to status=ASSIGNED, assignment_mode=TEAM; emits `WorkOrderAssigned`.

Step 3 — Notification fans out to team members. Three techs on the team get push notifications. `tech_01HZ_KAREN_03` claims via `POST /api/work-orders/wo_01HZ...Z1/claim` with `claimedByExternalDispatcher=false`. Framework checks: tech belongs to team ✓, has skill `INT + H01` ✓, schedule covers Mon 14:00 (per-tech row resolves to team default 09–18) ✓, no other non-terminal WO ✓. Atomic update wins; `assigned_technician_id=tech_01HZ_KAREN_03`, `claimed_at` set. Emits `WorkOrderClaimedByTechnician`. Other two techs' notifications cancelled.

Step 4 — Tech arrives at site, contractor dispatcher calls `/start`. Status → IN_PROGRESS, `in_progress_at` set, emits `WorkOrderInProgress`. BPMN advances to `on-site-work-loop`.

Step 5 — On-site work. Tech captures notes + attachments + equipment binding.

```
POST /api/work-orders/wo_01HZ...Z1/notes
{"noteKind":"findings","contentText":"Drop cable run from utility pole to wall plate completed. Cable modem powered on, CM

POST /api/work-orders/wo_01HZ...Z1/notes
{"noteKind":"hfc_signal_readings","contentJsonb":{"downstreamLevel":-3.2,"upstreamLevel":42.1,"snr":38.5,"package":"INET_5

POST /api/work-orders/wo_01HZ...Z1/notes
{"noteKind":"client_up_evidence","contentJsonb":{"evidenceType":"EASYDOCSIS_SCREENSHOT","comments":"Modem online on CMTS,

POST /api/work-orders/wo_01HZ...Z1/notes
{"noteKind":"solution","contentText":"Standard install completed. Customer signed off."}

POST /api/work-orders/wo_01HZ...Z1/notes
{"noteKind":"network_topology_confirmation","contentJsonb":{"confirmedAsPlanned":true}}

POST /api/work-orders/wo_01HZ...Z1/attachments  (multipart)
category=setup_photo, file=modem_wall_mount.jpg

POST /api/work-orders/wo_01HZ...Z1/attachments  (multipart)
category=speed_test, file=ookla_50_10.png

POST /api/work-orders/wo_01HZ...Z1/attachments  (multipart)
category=rf_optical_reading, file=easydocsis_cmts.png

POST /api/work-orders/wo_01HZ...Z1/equipment-bindings
{
  "bindings": [{
```

```
    "equipmentKind":"HFC_MODEM","brand":"Hitron","model":"CGNV4",
    "serialNumber":"HFC-MODEM-XYZ123","macAddress":"00:1A:2B:3C:4D:5E",
    "action":"INSTALLED","notes":"Mounted on living-room wall outlet"
  }],
  "recordedBy":"tech_01HZ_KAREN_03","recordedByRole":"FIELD_TECH"
}
```

Each call writes its row, emits the corresponding `WorkOrderNoteAppended` / `WorkOrderAttachmentAdded` / `WorkOrderEquipmentBindingsRecorded` event, audit-logs the action.

Step 6 — Contractor dispatcher first-confirms finalize.

```
POST /api/work-orders/wo_01HZ...Z1/finalize-first-confirm
{"finalReasonCode":"INSTALL_COMPLETED_CUSTOMER_UP","confirmedBy":"ctr_Z071H_dispatcher_01"}
```

Framework: status FINALIZATION_PENDING. Writes `final_reason_set` note. BPMN advances to `client-up-check` Drools gateway.

Step 7 — Drools runs `client-up-decision`. Inputs: `{jobTypeCode=H01, finalReasonCode=INSTALL_COMPLETED_CUSTOMER_UP, latestSignalReading={downstreamLevel:-3.2, snr:38.5}, attachmentCategories=[setup_photo, speed_test, rf_optical_reading]}`. Rule fires: thresholds met (downstream -3.2 ∈ [-15,15], SNR 38.5 ≥ 30), evidence note present, attachments cover required categories. Returns `CLIENT_UP`. BPMN takes yes-branch.

Step 8 — `wo-capture-equipment-bindings-at-finalize` worker runs. Reads `wo_equipment_ref_stub` for the WO. Job type H01 needs ≥1 `HFC_MODEM` action=INSTALLED. Found. Passes. Sets BPMN var `bindingsValidated=true`.

Step 9 — BPMN parks at `await-finalize-second-confirm`. Contractor dispatcher second-confirms.

```
POST /api/work-orders/wo_01HZ...Z1/finalize-second-confirm
```

Framework's `wo-enforce-finalize-checklist` reads `wo_finalization_requirements` for `(WIK, INSTALLATION, H01)`: required notes `[findings, solution, hfc_signal_readings, final_reason_set, client_up_evidence, network_topology_confirmation]` ✓; attachments `[setup_photo, speed_test, rf_optical_reading]` ✓. All present. Pass.

Framework: status=COMPLETED, completed_at set. Emits `WorkOrderFinalized` (framework event, with full equipment bindings array).

Step 10 — `wo-tag-warranty-window` worker runs. Reads catalog: H01 warranty 180 days. Computes `warranty_expires_at = 2026-11-14T17:45:00Z`. Updates `metadata` JSONB.

Step 11 — `wo-emit-homepass-update-at-finalize` worker runs. Reads the WO's `network_topology_confirmation` note. `confirmedAsPlanned=true` and no corrections. Emits `WorkOrderHomePassTopologyResolved` with `topologyMode=CONFIRMED_AS_PLANNED`. RLM-CFG-01 consumes it and audit-logs "topology confirmed at install" without writing changes to the HomePass record (planning data already matches reality).

Step 12 — Emits flow-specific `WorkOrderInstallationCompleted` event (§8.3 sample) carrying the equipment bindings, warranty window, and `topologyResolution={mode:"CONFIRMED",confirmedAsPlanned:true}`.

Step 13 — FUL-02 STEP-INSTALL consumes `WorkOrderFinalized`. Its consumer correlates on `originatingContextRef=ord_01HZ...A`, advances the FUL-02 BPMN, triggers activation via SUB-WF-ACTIVATE-01.

Total elapsed: ~6 hours (created 09:00, completed 14:50 + warranty/topology/install-completed events within seconds of finalize).

12.2 Pattern B reroute — GPON install with signal issue

Scenario. Customer John Otieno's GPON internet install. Tech arrives, ONT comes online but signal levels are below threshold.

Steps 1-5 identical to §12.1 but for `jobTypeCode=GIN` (GPON internet), Drools picks `ctr_GTECH + team_provision_westlands` (rule `r4-gpon-by-region`), tech `tech_01HZ_WESTLANDS_02` claims.

Step 6 — On-site work. Tech captures notes + optical readings.

```
POST /api/work-orders/wo_01HZ...B7/notes
{"noteKind":"findings","contentText":"ONT powered on, RX received, but signal weak"}

POST /api/work-orders/wo_01HZ...B7/notes
{"noteKind":"optical_readings","contentJsonb":{
  "oltPort":"0/11/1/6","onuId":12,"distanceM":1200,"opticalTempC":40,
  "ontRxDbm":-32.18,    // <-- BELOW threshold -28 dBm
```

```
  "ontTxDbm":2.01,"oltRxDbm":-9.54,"ip":"10.1.42.27","lastowncause":"LOS","batt":"notsupport"
}}
```

```
POST /api/work-orders/wo_01HZ...B7/equipment-bindings
{"bindings":[{"equipmentKind":"GPON_ONT","brand":"Huawei","model":"HG8245H",
  "serialNumber":"GPON-ONT-ABC789","action":"INSTALLED"}],
 "recordedBy":"tech_01HZ_WESTLANDS_02","recordedByRole":"FIELD_TECH"}
```

Step 7 — Tech first-confirms with success final reason (the success rule reflects tech's intent — they did install the ONT — but the gateway will catch it).

```
POST /api/work-orders/wo_01HZ...B7/finalize-first-confirm
{"finalReasonCode":"INSTALL_COMPLETED_CUSTOMER_UP","confirmedBy":"ctr_GTECH_dispatcher_02"}
```

Status → FINALIZATION_PENDING.

Step 8 — Drools `client-up-decision` **runs.** Inputs: `{jobTypeCode=GIN, finalReasonCode=INSTALL_COMPLETED_CUSTOMER_UP, latestOpticalReading={ontRxDbm:-32.18}, attachmentCategories=[setup_photo, rf_optical_reading]}`. Rule: GPON threshold check `ontRxDbm` ∈ `[-28, -8]`. `-32.18` is below `-28`. Returns `CLIENT_NOT_UP_REROUTE`. BPMN takes no-branch.

Step 9 — `wo-mark-pattern-b-reroute` **worker fires.**

1. Calls framework's `/finalize-revert` internally. Status → IN_PROGRESS. `final_reason_code` cleared.
2. Appends `pattern_b_reroute_reason` note:

   ```
   {"reasonCode":"SIGNAL_ISSUE","blockerDescription":"ONT RX power -32.18 dBm, below acceptable threshold -28 dBm. Need op
   ```

3. Determines next team: SIGNAL_ISSUE → `ctr_NOC_MAINT` / `team_noc_maintenance`.
4. Calls framework's `/reassign`:

   ```
   POST /api/work-orders/wo_01HZ...B7/reassign
   {
     "contractorId": "ctr_NOC_MAINT",
     "teamId":       "team_noc_maintenance",
     "technicianId": null,
     "scheduledAt":  "2026-05-19T09:00:00Z",
     "assignmentReason": "escalation_pattern_b"
   }
   ```

`wo_assignment_history` writes a row preserving the previous (GTECH/tech_02) and new (NOC/null) assignment. Emits `WorkOrderReassigned`.

5. Emits `WorkOrderReroutedPatternB` (§8.2 sample) with `originalInstallContractorId=ctr_GTECH`, `loopCount=1`.
6. BPMN loops back to `on-site-work-loop` with the new team owning the WO.

Step 10 — NOC investigates remotely. NOC opens the WO. Adds a `dispatcher_note`:

```
POST /api/work-orders/wo_01HZ...B7/notes
{"noteKind":"dispatcher_note","contentText":"Optical loss confirmed at OLT 0/11/1. Suspect splice damage between distribut
```

NOC reassigns to maintenance:

```
POST /api/work-orders/wo_01HZ...B7/reassign
{
  "contractorId":"ctr_NOC_MAINT","teamId":"team_maintenance_field",
  "technicianId":null,"assignmentReason":"manual_team_assignment"
}
```

Status stays IN_PROGRESS. Emits `WorkOrderReassigned`. `originalInstallContractorId` preserved in subsequent events.

Step 11 — Maintenance fixes the line. Maintenance tech goes to distribution point 47, re-splices, verifies fiber continuity. Adds notes + attachments:

```
POST /api/work-orders/wo_01HZ...B7/notes
{"noteKind":"findings","contentText":"Fiber splice at DP47 cracked due to weather. Re-spliced and reterminated. New OLT re
```

```
POST /api/work-orders/wo_01HZ...B7/notes
{"noteKind":"optical_readings","contentJsonb":{"ontRxDbm":-22.5,"oltRxDbm":-9.54}}
```

Maintenance reassigns the WO back to the install contractor for final attempt:

```
POST /api/work-orders/wo_01HZ...B7/reassign
{
  "contractorId":"ctr_GTECH","teamId":"team_provision_westlands",
  "technicianId":null,"assignmentReason":"return_to_originating_contractor"
}
```

Step 12 — Install contractor's tech returns to site. `tech_01HZ_WESTLANDS_02` claims again (they're free now; their prior WO is now in someone else's hands … wait, that's wrong — they're still the assigned tech of this WO until reassignment. Actually after reassignment they are released; framework R-WO-FW-11e check at re-claim time succeeds). Tech verifies ONT now reads `-22.5 dBm`, customer is up. Captures the new `optical_readings`, `client_up_evidence`, and a confirming setup photo.

Step 13 — Final attempt finalization. Tech first-confirms with same success reason. Drools `client-up-decision` runs against the latest optical reading (`-22.5` ∈ `[-28, -8]`): returns `CLIENT_UP`. Equipment bindings already present from step 6 — no need to re-record. `wo-capture-equipment-bindings-at-finalize` passes.

Second-confirm runs. Framework's `wo-enforce-finalize-checklist` verifies all required notes + attachments are present (multiple `findings` and `optical_readings` rows are fine — required check is ≥1 of each kind). Pass. Status=COMPLETED.

`WorkOrderFinalized` + `WorkOrderInstallationCompleted` emitted. The latter carries `patternBLoopCount=1`, `finalAssignedContractorId=ctr_GTECH`, `originalInstallContractorId=ctr_GTECH`. Warranty tagged: `warrantyExpiresAt=2026-11-19T15:30:00Z`.

`wo_assignment_history` now holds the full trail: `(GTECH/tech_02)` → `(NOC/-)` → `(NOC/maintenance)` → `(GTECH/-)` → `(GTECH/tech_02)`. Five rows. Useful for SLA analysis (Phase 2) and Finance (each team can be invoiced for their portion).

Total elapsed: ~26 hours (Day 1 install attempt 14:30, Day 2 maintenance 11:00, Day 2 final install 15:30).

12.3 Cross-country variation — YASSN install with fresh topology capture

Scenario. Customer Aïssatou Diop's GPON internet install in Dakar. Last mile is *not* pre-built — the install tech also runs the fiber from the PBO splitter to the customer's premise. The tech uses the field-tech mobile app ("app terrain") to capture the network topology on-site. All calls from the mobile app hit the same WO REST API the BO UI uses.

Step 1 — OSR-onboarding-SN (or FUL-02 equivalent for SN) creates the WO with `operatorCode=YASSN`, `kind=INSTALLATION`, `jobTypeCode=GIN`. Framework starts `WorkOrder_Installation_YASSN.bpmn`.

The HomePass record at this point holds only `{address, city, regionCode, plannedUpstreamNode}` — no `oltPort`, `pboId`, `splitterPort`, or `dropId`. The YASSN Drools KJAR's `contractor-assignment-decision` picks the regional contractor + team.

Steps 2–4 (assignment, claim, start) proceed as in §12.1. Tech `tech_01HZ_DKR_07` arrives at site.

Step 5 — Tech runs the last mile. Pulls fiber from the closest PBO (PBO-DKR-MAR-127) to the customer's premise, splices into splitter port 8, installs ONT inside. Customer is on splitter port 8 of PBO-DKR-MAR-127 and the OLT slot serving that PBO is `0/3/2`, port 4. Drop ID assigned by the construction crew: `DROP-887234`. Distance: 1450m.

Step 6 — Tech captures topology via mobile app. App terrain UI presents a form with the required topology fields. Tech fills it in and submits. The mobile app calls:

```
POST /api/work-orders/wo_01HZ...T9/notes HTTP/1.1
Authorization: Bearer <field-tech-svc-token>

{
  "noteKind":      "network_topology_capture",
  "contentJsonb": {
    "oltPort":         "0/3/2/4",
    "pboId":           "PBO-DKR-MAR-127",
    "splitterPort":    8,
    "dropId":          "DROP-887234",
    "distanceM":       1450,
    "plannedVsActualNotes": "Standard PBO splice. No deviations."
  },
  "appendedBy":     "tech_01HZ_DKR_07",
  "appendedByRole": "FIELD_TECH"
}
```

Framework: validates JSONB against the `YASSN.network_topology_capture` schema. All five required fields present. Writes `wo_notes` row, emits `WorkOrderNoteAppended`.

Step 7 — Tech captures optical readings, equipment binding, client-up evidence via the same mobile app (same REST calls as KE happy-path).

Step 8 — Tech captures attachments: setup photo, pbo_photo, rf_optical_reading, speed_test.

The `pbo_photo` attachment category is specific to YASSN's requirements — photographic evidence of the PBO splice work. Posted via the standard `POST /api/work-orders/{woId}/attachments` endpoint.

Step 9 — First-confirm finalize. Tech posts `finalReasonCode=INSTALL_COMPLETED_CUSTOMER_UP`. Status → FINALIZATION_PENDING.

Step 10 — Drools `client-up-decision` with YASSN-tuned thresholds runs. Returns `CLIENT_UP`. Equipment binding worker passes. Tech second-confirms.

Step 11 — Checklist runs. Required notes for `(YASSN, INSTALLATION, GIN)`: `[findings, solution, optical_readings, final_reason_set, client_up_evidence, network_topology_capture]` ✓; required attachment categories: `[setup_photo, pbo_photo, rf_optical_reading, speed_test]` ✓. All present. Pass.

Step 12 — Status=COMPLETED. `WorkOrderFinalized` emitted.

Step 13 — `wo-tag-warranty-window` runs (180 days from completed_at).

Step 14 — `wo-emit-homepass-update-at-finalize` **runs.** Reads the WO's `network_topology_capture` note. Resolves topology values: `oltPort=0/3/2/4`, `pboId=PBO-DKR-MAR-127`, `splitterPort=8`, `dropId=DROP-887234`, `distanceM=1450`. Emits `WorkOrderHomePassTopologyResolved` with `topologyMode=CAPTURED` (§8.4 sample). RLM-CFG-01 consumes and writes these five fields onto the HomePass record (previously NULL). The HomePass record now holds the as-installed topology.

Step 15 — Emits flow-specific `WorkOrderInstallationCompleted` carrying the equipment bindings, warranty window, and `topologyResolution={mode:"CAPTURED",oltPort:"0/3/2/4",pboId:"PBO-DKR-MAR-127",splitterPort:8,dropId:"DROP-887234",distanceM:1450}`.

Step 16 — OSR-onboarding-SN (or equivalent) consumes the event. Triggers provisioning via the SN-specific activation workflow. The provisioning system reads the now-populated HomePass topology and pushes the configuration to the OLT.

Key observation: same flow design, same code, same REST API, same Kafka events. What differed between §12.1 (WIK) and §12.3 (YASSN): the registered `note_kind`, the `wo_finalization_requirements` row, the Drools KJAR rules, the BPMN composition. All configuration; no code change. The downstream `WorkOrderHomePassTopologyResolved` event carries enough variation (`topologyMode`) for RLM-CFG-01 to handle both cases with the same consumer.

End of DD_WO-01-FLOW-INSTALLATION-v1.0.